WEEK- 5 : PAYMENT TRANSACTION MANAGEMENT SYSTEM

Introduction:

In an e-commerce system, payment information needs to be stored along with the orders placed by customers. A payment module helps the system record transaction details, identify the payment method used, and monitor whether a payment was successful, failed, or still pending.

In this week, a `PAYMENT` table was created and connected with the existing `ORDERS` table. Payment records were then added and SQL queries were used to analyze payment methods, revenue and customer payment history.

Payment Table Design :

```
SQL> CREATE TABLE PAYMENT (
  2      PAYMENT_ID VARCHAR2(7),
  3      ORDER_ID VARCHAR2(7),
  4      PAYMENT_DATE DATE,
  5      PAYMENT_MODE VARCHAR2(15),
  6      PAYMENT_STATUS VARCHAR2(10),
  7      TRANSACTION_AMOUNT NUMBER(10,2),
  8      PRIMARY KEY (PAYMENT_ID),
  9      FOREIGN KEY (ORDER_ID) REFERENCES ORDERS(ORDER_ID)
 10  );

Table created.

SQL> DESC PAYMENT;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL VARCHAR2(7)
 ORDER_ID                                           VARCHAR2(7)
 PAYMENT_DATE                                       DATE
 PAYMENT_MODE                                       VARCHAR2(15)
 PAYMENT_STATUS                                     VARCHAR2(10)
 TRANSACTION_AMOUNT                                 NUMBER(10,2)
```

```
SQL> SELECT * FROM PAYMENT;

PAYMENT ORDER_I PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PM001   O001    01-SEP-26 Card            Success                 2500
PM002   O002    02-SEP-26 UPI             Success                 1800
PM003   O003    03-SEP-26 Net Banking     Success                 3200
PM004   O004    04-SEP-26 Card            Failed                  1500
PM005   O005    05-SEP-26 UPI             Success                 4200
PM006   O006    06-SEP-26 Card            Success                 2750
PM007   O007    07-SEP-26 Net Banking     Success                 3600
PM008   O008    08-SEP-26 Cash            Pending                 1200
PM009   O009    09-SEP-26 UPI             Success                 5100
PM010   O010    10-SEP-26 Card            Success                 2300
PM011   O011    11-SEP-26 UPI             Success                 1900

PAYMENT ORDER_I PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PM012   O012    12-SEP-26 Net Banking     Pending                 3400
PM013   O013    13-SEP-26 Card            Success                 2800
PM014   O014    14-SEP-26 UPI             Success                 4500
PM015   O015    15-SEP-26 Cash            Pending                 1600

15 rows selected.
```

Report 1 — Display All Successful Payments :
The following query retrieves all payments whose status is `Success`.

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

PAYMENT ORDER_I PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PM001   O001    01-SEP-26 Card            Success                2500
PM002   O002    02-SEP-26 UPI             Success                1800
PM003   O003    03-SEP-26 Net Banking     Success                3200
PM005   O005    05-SEP-26 UPI             Success                4200
PM006   O006    06-SEP-26 Card            Success                2750
PM007   O007    07-SEP-26 Net Banking     Success                3600
PM009   O009    09-SEP-26 UPI             Success                5100
PM010   O010    10-SEP-26 Card            Success                2300
PM011   O011    11-SEP-26 UPI             Success                1900
PM013   O013    13-SEP-26 Card            Success                2800
PM014   O014    14-SEP-26 UPI             Success                4500

11 rows selected.
```

Report 2 — Find Failed Transactions :
Failed transactions can be retrieved using:

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Failed';

PAYMENT ORDER_I PAYMENT_D PAYMENT_MODE    PAYMENT_ST TRANSACTION_AMOUNT
------- ------- --------- --------------- ---------- ------------------
PM004   O004    04-SEP-26 Card            Failed                 1500
```

Report 3 — Count Total Successful and Failed Payments
The number of successful and failed transactions can be calculated using:

```
SQL> SELECT PAYMENT_STATUS,COUNT(*) AS TOTAL_PAYMENTS FROM PAYMENT WHERE PAYMENT_STATUS IN ('Success','Failed') GROU
P BY PAYMENT_STATUS;

PAYMENT_ST TOTAL_PAYMENTS
---------- -------------
Failed                1
Success              11
```

Payment Mode Analysis :

Payment methods used by customers were analyzed to understand which modes are used more frequently.

1. Number of UPI transactions

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS
  2  FROM PAYMENT
  3  WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSACTIONS
----------------
               5
```

This query gives the total number of transactions made using UPI.

2. Number of Card Payments

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS
  2  FROM PAYMENT
  3  WHERE PAYMENT_MODE = 'Card';

CARD_PAYMENTS
-------------
            5
```

This determines how many transactions were made using the Card payment method.

3. Most Preferred Payment Method

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUN
T DESC;

PAYMENT_MODE    TRANSACTION_COUNT
--------------- ----------------
Card                            5
UPI                             5
Net Banking                     3
Cash                            2
```

The payment mode with the highest transaction count can be considered the most frequently used payment method.

Revenue Analysis

The transaction amounts were analyzed to understand the revenue generated through successful payments.

1. Total Revenue Generated

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'Success';

TOTAL_REVENUE
-------------
        34650
```

This calculates the total amount received from successful transactions.

2. Revenue by Payment Method

```
SQL> SELECT PAYMENT_MODE,SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'Success'
  4  GROUP BY PAYMENT_MODE
  5  ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE    TOTAL_REVENUE
--------------- -------------
UPI                     17500
Card                    10350
Net Banking              6800
```

This query compares the revenue generated through different payment methods.

3. Average Transaction Amount

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT
  2  FROM PAYMENT
  3  WHERE PAYMENT_STATUS = 'Success';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                      3150
```

The query calculates the average value of successful payment transactions.

Customer Payment History

The payment information can also be combined with customer and order information to understand individual customer transactions.
The following query joins the `CUSTOMER`, `ORDERS` and `PAYMENT` tables:

```
SQL> SELECT c.CUSTOMER_NAME AS CUSTOMER_NAME,
  2         o.ORDER_ID AS ORDER_ID,
  3         p.PAYMENT_MODE AS PAYMENT_MODE,
  4         p.TRANSACTION_AMOUNT AS AMOUNT,
  5         p.PAYMENT_STATUS AS PAYMENT_STATUS
  6  FROM CUSTOMER c
  7  JOIN ORDERS o
  8  ON c.CUSTOMER_ID = o.CUSTOMER_ID
  9  JOIN PAYMENT p
 10  ON o.ORDER_ID = p.ORDER_ID
 11  ORDER BY p.PAYMENT_DATE;

CUSTOMER_NAME                                      ORDER_I PAYMENT_MODE        AMOUNT PAYMENT_ST
-------------------------------------------------- ------- --------------- ---------- ----------
Arun Kumar                                         O001    Card                  2500 Success
Priya Sharma                                       O002    UPI                   1800 Success
Arun Kumar                                         O003    Net Banking           3200 Success
Rahul Raj                                          O004    Card                  1500 Failed
Karthik M                                          O005    UPI                   4200 Success
Priya Sharma                                       O006    Card                  2750 Success
Anjali Devi                                        O007    Net Banking           3600 Success
Vijay Kumar                                        O008    Cash                  1200 Pending
Arun Kumar                                         O009    UPI                   5100 Success
Meeena K                                           O010    Card                  2300 Success
Ajay Kumar                                         O011    UPI                   1900 Success
Rahul Raj                                          O012    Net Banking           3400 Pending
Manoj R                                            O013    Card                  2800 Success
Nithya S                                           O014    UPI                   4500 Success
Karthik M                                          O015    Cash                  1600 Pending

15 rows selected.
```

Conclusion :

In this week, a Payment Transaction Management System was added to the existing e-commerce database.

The `PAYMENT` table was created and linked with the `ORDERS` table using a foreign key. 15 payment records were stored with different payment modes and transaction statuses.

SQL queries were then used to identify successful and failed transactions, analyze payment methods, calculate revenue and display customer payment history.

Thus, the payment module was successfully integrated with the existing e-commerce database and the required payment analysis was performed.